Learning Objectives

Learners will be able to...

- **Describe the fundamental principles of heap sort, including its structure and basic operations.**
- **Implement the heap sort algorithm, demonstrating the process of converting an unsorted array into a sorted sequence using heap properties.**
- **Analyze and calculate the time complexity of heap sort, explaining its efficiency in different scenarios and comparing it with other sorting algorithms.**
- **Compare heap sort with other common sorting algorithms in terms of performance, efficiency, and suitable use cases.**

Introduction to Heap Sort

Heap sort is a comparison-based sorting algorithm that uses a binary heap data structure. It's known for its efficiency and simplicity, especially when dealing with large data sets. The algorithm can be broken down into two main phases: building a heap and then sorting it.

Building a Heap

The first step in heap sorting is transforming the list or array into a binary heap. A binary heap can be a max-heap or a min-heap. In a max-heap, the parent node is always greater than or equal to the values of its children, while in a min-heap, the parent node is less than or equal to its children.

For heap sort, we typically use a max-heap. The process of creating a max-heap from an array is known as **heapifying down**. This process starts from the **last non-leaf** node and works upwards to the root, ensuring that each subtree satisfies the heap property.

Here's an example of turning an array into a max-heap:

Given array: `{ 3, 1, 6, 5, 2, 4 }`

The corresponding complete binary tree for this array (before heapifying) would be:

```
      3
     / \
    1   6
   / \ /
  5  2 4
```

Then, heapification occurs. Heapification here starts at the last non-leaf node. In this array, the last non-leaf node starts at index 2, which corresponds to the element `6`. To determine where the last non-leaf node is, you can apply this formula:

`last non-leaf node index = (total number of elements / 2) - 1` or `i = (n / 2) - 1`

In our case, `i` will be `(6 / 2) - 1` which equals `2`. Node `6` (index 2) is already greater than its children, so no change is needed. The next non-leaf node is node `1` at index 1. `1` is smaller than its children so we swap with the larger child `5` which results in:



The last non-leaf node is `3` at index 0 which is also the root. `3` is smaller than its children so it gets swapped with its larger child `6`:



But we're not done since `3` is still smaller than its child `4` now so we need to swap them:



All parent nodes are larger than their children nodes, so the max-heap is now complete: `{ 6, 5, 4, 1, 2, 3 }`

Sorting the Heap

Once the heap is formed, the sorting process begins. This involves repeatedly swapping the largest element (the root of the max-heap) with the last element in the array. Also, it's important to note that the root of the array is not completely removed from the array, rather it will go to the end of the array and will become the start of the **sorted** portion of the array. As the heap (unsorted portion of the array) shortens by one element, the sorted portion of the array lengthens by one element as well. After this swap, the heap properties are restored by heapifying down from the *new* root. Then, the next largest element is swapped again with the last unsorted element from the heap. This process continues until the heap is empty, which leaves the sorted part of the array fully "sorted."

Note two important details here: 1) The sorted portion of the array does not get involved in heapification and 2) Heapifying down starts at the *root* node during the sorting process rather than at the *last non-leaf* node (which occurred during building the heap).

Here's a step-by-step breakdown of the sorting process using the max-heap from above, note that the sorted portion of the array will be denoted with an asterisk (`*`):

Swap the root `6` with the last element in the heap `3`:



IMPORTANT: After the first root is swapped and ends up at the back of the array, the current index will decrease by one (`i - 1`). This will grow the sorted portion of the array while shrinking the heap. Index `i - 1` will take on the next largest element from the heap.

Resulting array: `{ 3, 5, 4, 1, 2, *6 }`

Restore the heap by heapifying down the tree starting from the new root node (`3`). `3` is smaller than its children so we swap it with its larger child `5`:



After that, the heap is restored. Next, swap the root `5` with the last unsorted element `2` which becomes the new root: `{ 2, 3, 4, 1, *5, *6 }`. Once again, the index will reduce by one.



Keep repeating the process (heapifying down, swapping the root with the last unsorted element, and reducing the index by one) until the heap is empty. The rest of the process looks like this.

Heapify:



Swap and reduce:

```
  1
 / \
3   2
```

Resulting array: `{ 1, 3, 2, *4, *5, *6 }`

Heapify:

```
  3
 / \
1   2
```

Swap and reduce:

```
  2
 /
1
```

Resulting array: `{ 2, 1, *3, *4, *5, *6 }`

Heapify (no swapping necessary):

```
  2
 /
1
```

Swap and reduce:

```
  1
```

Resulting array: `{ 1, *2, *3, *4, *5, *6 }`

The last element gets added to the sorted array automatically, no swapping or heapification occurs:

```
*empty heap
```

Resulting array: `{ *1, *2, *3, *4, *5, *6 }`

Once the heap is empty, heap sort ends and what remains is a sorted array.

Complexity and Efficiency

Heap sort has a time complexity of $O(nlogn)$ for both the worst and average cases, making it very efficient, especially for large datasets. Its space complexity is $O(1)$, as it sorts the array in place.

This lesson has introduced the basics of heap sort, including its process and efficiency. In the upcoming lessons, we'll explore the implementation and compare heap sort with other sorting algorithms to understand its advantages and use cases better. Next, we'll delve into the implementation details of heap sort, providing a step-by-step guide to coding the algorithm.

Implementing Heap Sort

Heap sort implementation involves two main functions: building a heap (usually a max-heap) and then sorting it. Let's walk through the implementation step by step.

Step 1: Building a Max-Heap

First, we need to convert the given array into a max-heap. We do this by calling the `heapify` method for each non-leaf node, starting from the last non-leaf node and moving upwards.

Here's how you can implement the `heapify` method:

```
void heapify(int array[], int n, int i) {
    int largest = i; // Initialize largest as root
    int left = 2 * i + 1; // left child
    int right = 2 * i + 2; // right child

    // If left child is larger than root
    if (left < n && array[left] > array[largest])
        largest = left;

    // If right child is larger than largest so far
    if (right < n && array[right] > array[largest])
        largest = right;

    // If largest is not root
    if (largest != i) {
        int swap = array[i];
        array[i] = array[largest];
        array[largest] = swap;

        // Recursively heapify the affected sub-tree
        heapify(array, n, largest);
    }
}
```

Step 2: Sorting the Heap

Once the max-heap is built, we can start sorting the array. We repeatedly move the root of the heap (the largest element) to the end of the array, reduce the heap size, and then heapify the root to maintain the max-heap property.

Here's the complete implementation of heap sort:

```
void heapSort(int array[], int n) {
    // Build heap (rearrange array)
    for (int i = n / 2 - 1; i >= 0; i--)
        heapify(array, n, i);

    // Extract elements from heap one by one
    for (int i = n - 1; i > 0; i--) {
        // Move current root to end
        int temp = array[0];
        array[0] = array[i];
        array[i] = temp;

        // Call max heapify on the reduced heap
        heapify(array, i, 0);
    }
}
```

Visualizing Heap Sort

Let's visualize the heap sort process with another example:

Given array: `{ 7, 12, 9, 4, 11, 6 }`

1. **Build Max-Heap:**

```
    7
   / \
  12  9
 / \  /
4  11 6
```

   - Heapify process begins at the last non-leaf node, which is 9.
   - 9 is already greater than its child so no swap is necessary.
   - Move onto the next non-leaf node up, 12.
   - 12 is also greater than its children so no swap is needed.
   - Move onto the next non-leaf node, 7 (also the root node).
   - 7 is smaller than its children, so swap with the larger child 12.

```
    12
   / \
  7   9
 / \  /
4  11 6
```

   - 7 is now smaller than one of its children 11 so a swap is performed between them.

```
   12
   / \
  11  9
 / \ /
4  7 6
```

- Max-heap is now complete.

2. **Sort the Heap:**
   - Swap `12` (root) with `6` (last element), reduce the heap size by 1, and heapify:

     ```
      11
      / \
     7   9
    / \
   4   6
     ```

     Sorted array: `[12]`

   - Continue the process:

     ```
       9
      / \
     7   6
    /
   4
     ```

     Sorted array: `[11, 12]`

   - And so on, until the heap is empty.

Testing Heap Sort

Let's test our heap sort implementation with a `main` method.

```
int main() {
    HeapSortImplementation sorter;
    int array[] = {7, 12, 9, 4, 11, 6};
    int n = sizeof(array) / sizeof(array[0]);

    std::cout << "Original array: ";
    for (int i = 0; i < n; i++)
        std::cout << array[i] << " ";
    std::cout << std::endl;

    sorter.heapSort(array, n);

    std::cout << "Sorted array: ";
    for (int i = 0; i < n; i++)
        std::cout << array[i] << " ";
    std::cout << std::endl;

    return 0;
}
```

Add this `main` method to your student file to test the heap sort algorithm.

Student's Code File

Here is the complete code for your `HeapSortImplementation.cpp` file:

```
class HeapSortImplementation  {
public:
void heapify(int array[], int n, int i) {
    int largest = i; // Initialize largest as root
    int left = 2 * i + 1; // left child
    int right = 2 * i + 2; // right child

    // If left child is larger than root
    if (left < n && array[left] > array[largest])
        largest = left;

    // If right child is larger than largest so far
    if (right < n && array[right] > array[largest])
        largest = right;

    // If largest is not root
    if (largest != i) {
        int swap = array[i];
        array[i] = array[largest];
        array[largest] = swap;

        // Recursively heapify the affected sub-tree
        heapify(array, n, largest);
    }
}

    void heapSort(int array[], int n) {
```

```cpp
        // Build heap (rearrange array)
        for (int i = n / 2 - 1; i >= 0; i--)
            heapify(array, n, i);

        // Extract elements from heap one by one
        for (int i = n - 1; i > 0; i--) {
            // Move current root to end
            int temp = array[0];
            array[0] = array[i];
            array[i] = temp;

            // Call max heapify on the reduced heap
            heapify(array, i, 0);
        }
    }

};

int main() {
    HeapSortImplementation sorter;
    int array[] = {7, 12, 9, 4, 11, 6};
    int n = sizeof(array) / sizeof(array[0]);

    std::cout << "Original array: ";
    for (int i = 0; i < n; i++)
        std::cout << array[i] << " ";
    std::cout << std::endl;

    sorter.heapSort(array, n);

    std::cout << "Sorted array: ";
    for (int i = 0; i < n; i++)
        std::cout << array[i] << " ";
    std::cout << std::endl;

    return 0;
}
```

Time Complexity Analysis of Heap Sort

Heap sort is a comparison-based sorting technique based on a binary heap data structure. Its efficiency lies in its ability to sort data in O(n log n) time complexity. Let's break down this complexity analysis.

Building the Heap

The first step in heap sort is building a heap from the input data. This process involves arranging the elements in a way that satisfies the heap property. For a max heap, this means the parent node is always larger than its children.

Complexity of Heapify

The `heapify` function, which is used to maintain the heap property, has a time complexity of O(log n) for a single call. This is because the height of a binary heap is log n, and in the worst case, `heapify` might need to traverse from the root to the deepest leaf node.

Complexity of Building the Heap

Building the heap involves calling `heapify` for each non-leaf node. There are approximately n/2 non-leaf nodes in a complete binary tree. However, not every `heapify` call traverses the full height of the tree. On average, the time complexity for building the heap is O(n).

Sorting the Heap

Once the heap is built, the sorting process involves repeatedly removing the root of the heap (the largest element) and then calling `heapify` to restore the heap property. This process is repeated n times.

Complexity of Sorting

Each removal of the root element and subsequent `heapify` call has a time complexity of O(log n). Since there are n such removals, the total time complexity for the sorting phase is O(n log n).

Total Time Complexity

The total time complexity of heap sort is the sum of the complexities of building the heap and sorting it. Therefore, the overall time complexity is O(n) + O(n log n), which simplifies to O(n log n).

Visualizing Time Complexity

Consider the following heap:



1. **Building the Heap:** O(n) time complexity.
2. **Sorting the Heap:** Each removal and `heapify` takes O(log n) time, repeated n times, leading to O(n log n) time complexity.

Conclusion

Heap sort's time complexity of O(n log n) makes it an efficient sorting algorithm, especially for large datasets. Its ability to sort in place, without requiring additional memory, adds to its practicality. In the next lesson, we will compare heap sort with other sorting algorithms to understand its relative strengths and weaknesses.

Heap Sort vs. Other Sorting Algorithms

Heap sort is a popular sorting algorithm, but how does it stack up against other common sorting methods like quick sort, merge sort, and insertion sort? Let's compare their time complexities, stability, and typical use cases.

Quick Comparison Table

Sorting Algorithm	Best Case	Average Case	Worst Case	Stability	In-Place
Heap Sort	$O(nlogn)$	$O(nlogn)$	$O(nlogn)$	No	Yes
Quick Sort	$O(nlogn)$	$O(nlogn)$	$O(n^2)$	No	Yes
Merge Sort	$O(nlogn)$	$O(nlogn)$	$O(nlogn)$	Yes	No
Insertion Sort	$O(n)$	$O(n^2)$	$O(n^2)$	Yes	Yes

Heap Sort

Heap sort is known for its consistent $O(nlogn)$ time complexity in all cases. It's not a stable sort, meaning it doesn't maintain the relative order of equal elements. However, it sorts in place, requiring only a constant amount of additional space.

Example: Heap Sort in Action

Consider sorting the following array using heap sort:

```
Array: [5, 3, 17, 10, 84, 19, 6, 22, 9]
```

Heap sort first transforms this array into a heap and then sorts it, maintaining the $O(nlogn)$ complexity throughout.

Quick Sort

Quick sort is often faster in practice, thanks to its $O(nlogn)$ average time complexity. However, its worst-case scenario is $O(n^2)$, which can occur with certain pivot choices. Like heap sort, it's not stable but sorts in place.

Example: Quick Sort Pivot

In a quick sort, choosing the right pivot is crucial. A bad pivot choice can lead to the $O(n^2)$ worst-case scenario.

Merge Sort

Merge sort guarantees an $O(nlogn)$ time complexity in all cases and is stable. However, it's not an in-place sorting algorithm and requires additional space, which can be a drawback for large datasets.

Example: Merge Sort Splitting

Merge sort divides the array into halves, sorts each half, and then merges them. This process requires additional space for the merging phase.

Insertion Sort

Insertion sort is efficient for small datasets or nearly sorted arrays, with a best-case time complexity of $O(n)$. However, its average and worst-case time complexities are $O(n^2)$, making it inefficient for large datasets. It's stable and sorts in place.

Example: Insertion Sort with Nearly Sorted Array

Insertion sort performs exceptionally well with nearly sorted arrays, often approaching $O(n)$ complexity.

Conclusion

Heap sort is a reliable choice for consistent $O(nlogn)$ performance, especially when stability is not a concern, and additional space is limited. In contrast, quick sort and merge sort might be preferred in scenarios where average performance and stability are more critical, respectively. Understanding these differences helps in selecting the most suitable sorting algorithm for a given problem.

Formative Assessment 1

Formative Assessment 2